

# Anti-PE LightGBM Agent Scanner: Codebase Explanation

## 1. What This Project Is

This codebase is an inference-only Windows PE malware scanner intended for Agent-side integration.

It takes either a file path or a watched directory as scanner input. It does not send raw files directly into the model. Instead, it validates that a file is a Windows PE file, extracts the exact 2381 numerical PE features expected by the trained model, scores those features with a pre-trained LightGBM model, applies policy thresholds, and emits a JSON event.

The project is deliberately narrow:

- LightGBM only
- PE static-feature inference only
- Single-file scan mode
- Continuous directory watch mode
- JSON output
- No training
- No XGBoost
- No stacking or meta-classifier
- No hard mining
- No SHAP, plots, notebooks, or research artifacts
- No EMBER dataset files
- No malware samples

## 2. Mental Model

The most important design boundary is this:

```
"""text
Scanner input: file path or watch directory
Model input: 2381 numerical PE features
Model output: malware probability score
Scanner output: JSON event
"""
```

The raw file is never the model input. The scanner is a pipeline that safely converts a file path into a structured model-ready vector, then turns the model score into a policy decision.

## 3. High-Level Lifecycle

For a single file, the lifecycle is:

```
"""text
1. CLI or caller passes a file path
2. Scanner loads policy and model package
3. Scanner checks whether scanning is enabled
4. PE validator checks file existence, size, MZ header, PE header offset, and PE signature
5. Non-PE or invalid files are skipped and returned as JSON
6. Valid PE files go to feature extraction
7. Feature extractor uses LIEF and Anti_PE ((2))-derived EMBER-style feature logic
8. Feature vector is validated as shape (1, 2381)
9. LightGBM Booster scores the vector
"""
```

10. Policy thresholds convert the score into allow/log/alert/block
  11. Event builder emits a JSON-serializable event
- ""

For watch mode, the lifecycle is:

- ""text
1. CLI starts a watchdog observer for a directory
  2. Created or modified file events are observed
  3. Watch helper waits until file size is stable
  4. Duplicate unchanged events are suppressed
  5. Each stable file path is sent to the same single-file scanner
  6. One JSON event is printed per scanned file
  7. Process keeps running until Ctrl+C
- ""

## 4. Project Structure

""text

```
anti-pe-lightgbm-agent-scanner/  
  src/anti_pe_scanner/  
    errors.py  
    schemas.py  
    policy.py  
    verdict.py  
    utils.py  
    event_builder.py  
    model_loader.py  
    pe_validator.py  
    feature_extractor.py  
    scanner.py  
  
  tools/  
    scan_file.py  
    watch_dir.py  
    build_exe.py  
  
  models/lightgbm_pe_v1/  
    model.txt  
    metadata.json  
    thresholds.json  
    feature_config.json  
  
  configs/  
    policy.json  
  
  tests/  
    test_*.py  
  
  docs/  
    HOW_TO_RUN.md  
    MODEL_DELIVERY.md  
    INTEGRATION_NOTES.md  
    EVENT_SCHEMA.md  
    CONTINUOUS_MODE.md
```

HANDOVER\_SUMMARY.md  
CODEBASE\_EXPLAINED.md  
CODEBASE\_EXPLAINED.pdf

dist/  
  anti\_pe\_scanner  
  models/  
  configs/  
  docs/  
'''

## 5. Core Runtime Modules

'src/anti\_pe\_scanner/errors.py'

This file defines shared string constants for scan statuses, verdicts, and actions.

Scan statuses include:

- 'success'
- 'policy\_disabled'
- 'skipped\_not\_pe'
- 'skipped\_too\_large'
- 'file\_not\_found'
- 'read\_error'
- 'parse\_error'
- 'feature\_error'
- 'model\_error'

Verdicts include:

- 'allow'
- 'log'
- 'alert'
- 'block'

Actions include:

- 'none'
- 'block'

The scanner uses these constants so every module speaks the same status language and JSON consumers can rely on stable values.

'src/anti\_pe\_scanner/schemas.py'

This file contains simple dataclasses used throughout the scanner:

- 'PolicyConfig'
- 'FileInfo'
- 'ModelInfo'
- 'DecisionInfo'
- 'ScanError'
- 'ScanResult'
- 'ValidationResult'

These dataclasses are intentionally JSON-friendly. They hold plain values such as strings, numbers, booleans, dictionaries, and 'None'.

The helper 'to\_jsonable()' converts dataclasses and containers into plain Python structures suitable for JSON serialization.

'src/anti\_pe\_scanner/policy.py'

This file owns policy loading, merging, and validation.

The default policy is:

```
"""json
{
  "scan_enabled": true,
  "mode": "alert_only",
  "alert_threshold": 0.6242529630641415,
  "block_threshold": 0.9385114343804422,
  "max_file_size_mb": 100,
  "quarantine_enabled": false
}
"""
```

Valid policy modes are:

- 'log\_only'
- 'alert\_only'
- 'block\_enabled'

Validation rules:

- thresholds must be between 0 and 1
- alert threshold must be less than block threshold
- max file size must be positive
- unknown policy modes raise 'ValueError'

The scanner merges the base policy, model package thresholds, and optional user override policy.

'src/anti\_pe\_scanner/verdict.py'

This file turns a model score into a decision.

Decision rules:

- score below alert threshold: verdict 'allow', action 'none'
- score between alert and block thresholds:
  - 'log\_only': verdict 'log'
  - 'alert\_only': verdict 'alert'
  - 'block\_enabled': verdict 'alert'
- score above block threshold:
  - 'log\_only': verdict 'log'
  - 'alert\_only': verdict 'alert'
  - 'block\_enabled': verdict 'block', action 'block'

Important: scanner v1 only reports the action in JSON. It does not quarantine, delete, or directly block files.

'src/anti\_pe\_scanner/utils.py'

This file contains small utility functions:

- 'sha256\_file(path)'
- 'utc\_now\_iso()'
- 'safe\_json\_dumps(data, pretty=False)'
- 'file\_size\_bytes(path)'
- 'is\_file\_size\_stable(path, checks=2, interval\_sec=0.5)'

Watch mode uses 'is\_file\_size\_stable()' so it does not scan files while they are still being copied or written.

'src/anti\_pe\_scanner/event\_builder.py'

This file builds the final JSON event.

Every event includes:

```
"""json
{
  "event_type": "ml_pe_scan",
  "event_version": "1.0",
  "timestamp": "...",
  "scan_status": "...",
  "file": {},
  "model": {},
  "decision": {},
  "error": null
}
"""
```

If host context is passed by an Agent integration, it is included under 'host\_context'.

'src/anti\_pe\_scanner/model\_loader.py'

This file loads the LightGBM model package.

It requires:

- 'model.txt'
- 'metadata.json'
- 'thresholds.json'
- 'feature\_config.json'

It validates:

- model type is 'lightgbm'
- feature dimension is '2381'
- thresholds are valid
- alert threshold is less than block threshold

It loads the trained model with:

```
"""python
lightgbm.Booster(model_file=str(model_path))
"""
```

No training is performed. The model is already trained.

'predict\_score(features)' accepts either:

- shape '(2381,)'
- shape '(1, 2381)'

It returns a Python 'float'.

'src/anti\_pe\_scanner/pe\_validator.py'

This file decides whether a file is safe and valid enough to send to feature extraction.

It checks:

- file exists
- path is a regular file
- file is readable
- file is non-empty
- file size does not exceed policy limit
- file starts with 'MZ'
- DOS header contains a PE header offset at '0x3C'
- PE signature at that offset is 'PE\0\0'
- optional LIEF parse if LIEF is installed

Non-PE files never reach feature extraction or model inference.

'src/anti\_pe\_scanner/feature\_extractor.py'

This file bridges a valid PE file to the model input.

It adapts the feature extraction logic from Anti\_PE ((2)) and preserves the feature group order expected by the trained model.

Feature groups:

```
"""text
ByteHistogram[0:256]
ByteEntropyHistogram[256:512]
StringFeatures[512:616]
GeneralFileInfo[616:626]
HeaderFileInfo[626:688]
SectionInfo[688:943]
ImportsInfo[943:2223]
ExportsInfo[2223:2351]
DataDirectories[2351:2381]
"""
```

Output:

- NumPy array
- shape '(1, 2381)'
- 'float32'
- no NaN or infinite values

If extraction fails, it raises 'FeatureExtractionError'. It does not create fake feature vectors.

'src/anti\_pe\_scanner/scanner.py'

This is the single-file orchestration layer.

'PEMalwareScanner' does the full scan flow:

```
"""text
policy -> validation -> feature extraction -> model score -> verdict -> JSON event
"""
```

It catches bad inputs and runtime failures and returns structured JSON events instead of crashing.

It does not call model inference for invalid or non-PE files.

## 6. Tooling Modules

'tools/scan\_file.py'

This is the CLI entry point.

Single-file mode:

```
"""bash
PYTHONPATH=src .venv/bin/python tools/scan_file.py --file README.md --model-package
models/lightgbm_pe_v1 --pretty
"""
```

Watch mode:

```
"""bash
PYTHONPATH=src .venv/bin/python tools/scan_file.py --watch-dir ./samples --model-package
models/lightgbm_pe_v1 --recursive --pretty
"""
```

The same entry point is used by PyInstaller.

'tools/watch\_dir.py'

This file implements continuous watch mode with 'watchdog'.

It:

- monitors created and modified files
- waits for file size to stabilize

- avoids duplicate scans for unchanged files
- prints one JSON event per scanned file
- keeps running after per-file errors
- stops cleanly with Ctrl+C

Watch mode is a v1 operational adapter. Final Agent integration may call the scanner per Agent file event instead.

'tools/build\_exe.py'

This file builds the PyInstaller package.

It uses 'tools/scan\_file.py' as the entry point and keeps the model package beside the executable.

The staged layout is:

```
"""text
dist/
  anti_pe_scanner or anti_pe_scanner.exe
  models/lightgbm_pe_v1/
  configs/policy.json
  docs/
"""
```

On Linux, PyInstaller creates 'anti\_pe\_scanner'. On Windows, build in a Windows environment to create 'anti\_pe\_scanner.exe'.

## 7. Model Package

Location:

```
"""text
models/lightgbm_pe_v1/
"""
```

Files:

- 'model.txt': trained LightGBM model
- 'metadata.json': model identity and feature metadata
- 'thresholds.json': alert and block thresholds
- 'feature\_config.json': feature dimension and extractor warning

Important values:

```
"""text
model_name: lightgbm_pe_malware_detector
model_version: v1
model_type: lightgbm
feature_dim: 2381
alert_threshold: 0.6242529630641415
block_threshold: 0.9385114343804422
"""
```

## 8. Configuration



'configs/policy.json' is the default runtime policy.

It controls:

- whether scanning is enabled
- mode: 'log\_only', 'alert\_only', or 'block\_enabled'
- alert threshold
- block threshold
- maximum file size
- quarantine flag

The current scanner does not perform quarantine. The flag is present for policy compatibility and future Agent-side enforcement.

## 9. Tests

The test suite covers:

- policy validation
- verdict mapping
- event JSON shape
- model loading
- PE validation
- feature-vector validation
- scanner behavior on missing/non-PE/fake/real benign PE files
- CLI behavior
- watch-mode helper behavior

Current validation result:

```
"""text
45 passed
"""
```

## 10. Packaging and Distribution

Build on Linux/source:

```
"""bash
PYTHONPATH=src .venv/bin/python tools/build_exe.py
"""
```

Build Windows executable on Windows:

```
"""powershell
.venv\Scripts\python.exe tools\build_exe.py
"""
```

The model package must stay beside the executable.

Do not move only the binary by itself. Move the whole 'dist/' layout.

## 11. What Happens for Common Inputs

Missing file

Result:

```
""text
scan_status = file_not_found
error != null
""
```

Non-PE text file

Result:

```
""text
scan_status = skipped_not_pe
file.type = non_pe
model.score = null
decision = null
""
```

Valid benign PE

Result:

```
""text
scan_status = success
file.type = windows_pe
model.score = number between 0 and 1
decision.verdict = allow/log/alert/block
error = null
""
```

## 12. Known Limitations

- Windows '.exe' must be built on Windows or in a Windows build environment.
- Current real-PE validation used one benign PE sample.
- Golden-vector parity testing against the old Anti\_PE ((2)) extractor is recommended before production deployment.
- Scanner v1 reports block actions in JSON but does not directly quarantine, delete, or block files.
- Final Agent integration may replace folder watching with Agent-provided file events.

## 13. Recommended Agent Integration Path

Recommended handoff sequence:

1. Agent receives or observes a file path.
2. Agent calls scanner module or executable with that path.
3. Scanner returns JSON.
4. Agent records telemetry.
5. Agent decides whether to enforce action based on policy and 'decision.action'.
6. Agent handles any quarantine/delete/block behavior outside scanner v1.

This keeps the ML scanner small, inspectable, and focused on inference.